MACHINE LEARNING AND DEEP LEARNING

Lab 01: Python and NumPy

Dr. Giacomo Lucato, a.y. 2025-2026

LAB SESSIONS ORGANIZATION

- Material will be uploaded on Moodle page shortly before the lab session
- Review of exercises of the previous lab sessions
- Theoretical introduction about new lab topics
- New coding exercises will be held to you to be solved alone or in groups

- For any issue or question, write me an email at any time: giacomo.lucato@studenti.univr.it

PYTHON

- Python is a **high-level, versatile, cross platform programming language,** widely used for scientific computing, especially Machine Learning and Deep Learning.
- Python provides a **high-level abstraction** for low-level, high-performance compute libraries like TensorFlow, PyTorch and Keras.
- It integrates **specialized libraries** for data manipulation (NumPy, Pandas, …) with **hardware-optimized backends** that leverage both CPU and GPU architectures.
- **IDEs:** VSCode or online tools such as Google Colab (which provides free yet limited GPU access).
- **Important:** turning off AI Coding Agents is strongly recommended.

VIRTUAL ENVIRONMENTS

- If you choose to work locally (e.g., on VSCode), you should create a **virtual environment** for these lab lectures.
- You can choose either Conda or venv.
- For Conda:
  - Install Conda: https ://www.anaconda.com/products/individual#Downloads
  - Create a virtual env:
    - $ conda create --name <env_name> python=3.10
  - Activate the env:
    - $ conda activate <env_name>
  - Install dependencies:
    - $ pip install –r requirements.txt
  - Once finished working, deactivate the env:
    - $ conda deactivate

VIRTUAL ENVIRONMENTS

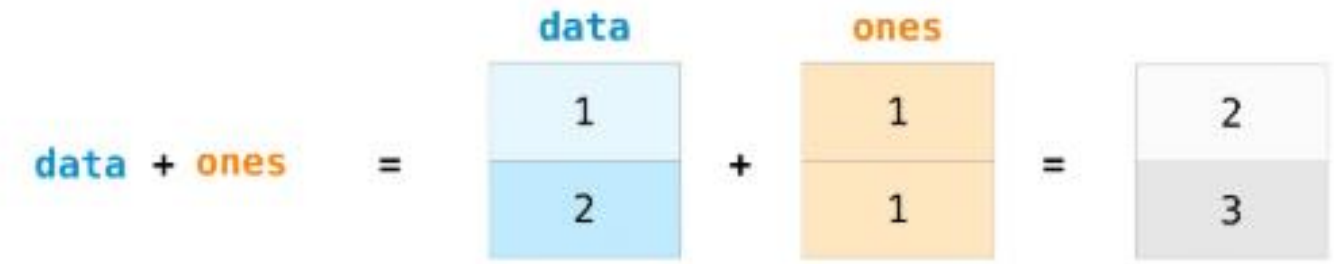
- For venv:
  - Create an env:
    - $ python -m venv <env_name>
  - Activate the env:
    - Windows: $ env_name\Scripts\activate
    - Linux: $ source env_name/bin/activate
  - Install dependencies:
    - $ pip install -r requirements.txt
  - Once done, deactive the env:
    - $ deactivate

NUMPY

- **NumPy** is a fundamental python library for **scientific computing,** providing high-performance tools for manipulating large, **multi-dimensional arrays** and matrices.
- The core of the NumPy library is the ***ndarray*** data structure, a n-dimensional array stored contiguously in memory, allowing for faster and more efficient operations.
- **Vectorization:** in standard python, loops are slow and inefficient. NumPy provides the so-called ***vectorization trick*** that allows to perform operations over arrays without relying on slow loops.

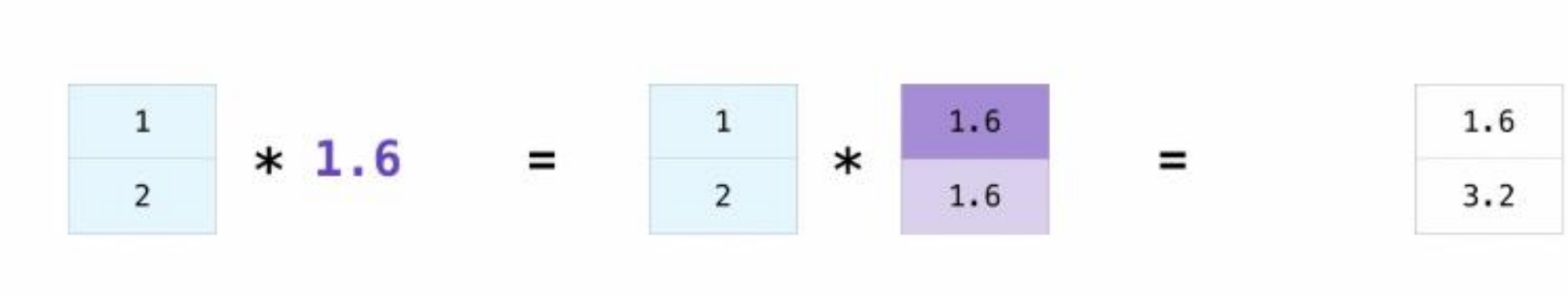
NUMPY - VECTORIZATION

- Vectorization is the main reason one should use NumPy over standard python lists.



NUMPY - BROADCASTING

- A consequence of vectorization is **broadcasting:** it allows fast and efficient operations between arrays of different sizes (e.g., multiplying a scalar for a 1d-vector).



NUMPY - EXAMPLES

- `>>> import numpy as np  # importing numpy`
- `>>> data = np.array([1, 2])  # creating and initializing a 1d-array`
- `>>> n = data[2]    # accessing the third element of an array`
- `>>> a = np.array([[1, 2, 3], [5, 6, 7], [9, 10, 11]])  # creating a 3x3 matrix`
- `>>> print(a.ndim)  # print the number of dimensions (axis) of an array`
- `>>> print(a.shape) # print the shape: tuple containing the number of elements across each axis`

NUMPY - EXAMPLES

- `>>> x = np.ones(2, dtype=np.int64)  # creating an array of ones`
- `>>> arr1 = a[3:8]  # accessing elements from position 3 (included) to position 8 (excluded) to create a new array`
- `>>> arr1 * 1.6  # broadcasting: multiplying an array for a scalar`
- `>>> a1 = np.array([1, 2, 3])`
- `>>> a2 = np.array([4, 5, 6])`
- `>>> r = a1 + a2  # summing two arrays`
- `>>> s = r.sum()  # summing over all elements of an array`

EXERCISES

- Download the folder Lab_01 from Moodle
- Open the Lab_01.ipynb file and complete the code following the instructions in the comments
- Useful NumPy documentation: NumPy: the absolute basics for beginners — NumPy v2.4 Manual
- Exercises:
  - Ex 1: basic array operations
  - Ex 2: basic matrix operations
  - Ex 3: computing Euclidean distance between two sets of 2d-points
  - Ex 4: image manipulation through NumPy